

深度学习基本概念简介

李宏毅《机器学习》2025 · 第一节（下）

基于李宏毅老师课程整理

2026 年 6 月 4 日

视频信息

频道：李宏毅深度学习课堂（Bilibili 搬运）

时长：约 58 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=2>

目录

1 回顾：线性模型的局限	3
2 用分段线性曲线逼近任意函数	4
2.1 Piecewise Linear Curve (分段线性曲线)	4
2.2 Sigmoid 函数——从离散到平滑	4
2.3 多特征扩展模型	6
3 模型的矩阵/向量表示	7
3.1 网络结构图	7
3.2 线性代数表示	8
4 损失函数与梯度下降优化	9
4.1 损失函数 $L(\theta)$	9
4.2 梯度下降的向量形式	9
4.3 Batch 训练: Epoch 与 Update	10
5 ReLU——另一种激活函数	11
6 从浅层到深层——深度学习	12
6.1 多层堆叠 (Deep Stacking)	12
6.2 神经网络的命名学	12
7 实验结果	13
7.1 ReLU 数量与深度的实验	13
7.2 预测效果可视化	14
8 过拟合与模型选择	14
8.1 什么是过拟合 (Overfitting)	14
8.2 模型选择的原则	15
8.3 关于“为什么是 Deep 而不是 Fat”	15
9 本章小结	16
9.1 下一讲预告	17

1 回顾：线性模型的局限

在上一节课中，我们介绍了机器学习的基本框架：定义一个带有未知参数的函数（Step 1）、定义损失函数（Step 2）、优化参数使损失最小（Step 3）。我们用线性模型 $y = b + wx_1$ 完成了三步流程。

但线性模型有一个本质性的限制——**Model Bias**（模型偏差）。注意：这里的“Model Bias”与参数 b （y 轴截距，也叫 bias）是不同的概念。Model Bias 指的是模型本身的结构限制了它能表达的函数空间。

Model Bias（模型偏差）

模型偏差（Model Bias）指的是：由于模型形式的限制，无论你怎么调整参数，都无法准确模拟真实的数据关系。它是模型结构本身的局限性，与参数 b （截距/bias）是不同的概念。

如图 1 所示，线性模型只能表示一条直线： x_1 越大， y 就越大。但现实中的数据关系往往不是这样。可能 x_1 小于某个值时， y 随 x_1 增大；而 x_1 超过某个峰值后， y 反而下降（“物极必反”）。无论你怎么调整 w 和 b ，永远制造不出那条红色的复杂曲线。

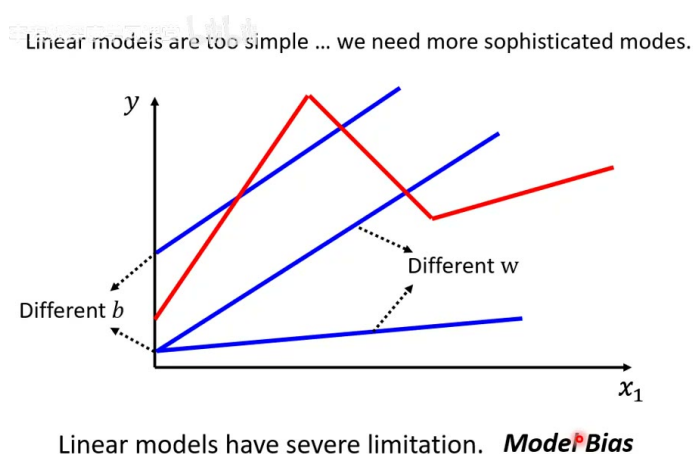


图 1: 线性模型（蓝色直线）无法拟合真实的数据关系（红色曲线），这种限制称为 Model Bias。

模型能力与偏差

线性模型 $y = b + wx_1$:

- 只能表示 x_1 与 y 之间的直线关系
- x_1 越大 $\rightarrow y$ 越大（单调关系）
- 无法表示非线性、非单调的关系

这就产生了 Model Bias——由于模型形式过于简单，无法拟合真实世界的复杂规律。

2 用分段线性曲线逼近任意函数

2.1 Piecewise Linear Curve (分段线性曲线)

我们能否写出一个更复杂、更有弹性的函数来减少 Model Bias?

关键观察：红色的复杂曲线可以看作是一个**常数**加上一群**蓝色函数**的组合。这个蓝色函数的特点是：输入小于某个阈值时是水平线，大于另一个阈值时也是水平线，中间有一个斜坡——我们暂且称它为“蓝色函数”（它的正式学名是 Hard Sigmoid）。

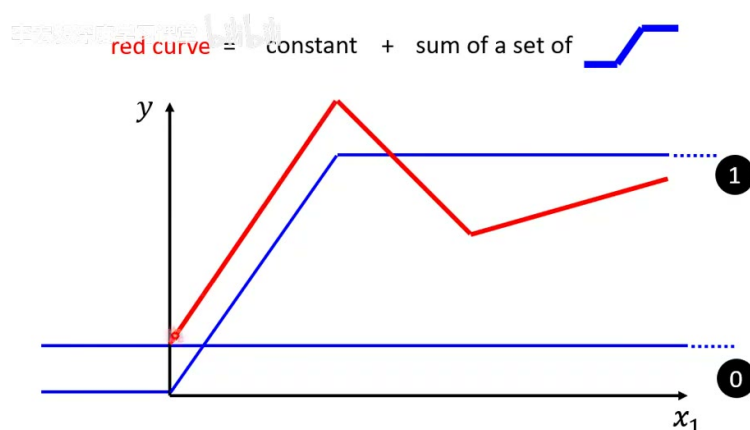


图 2: 红色曲线 = 常数项 + 蓝色函数 1 + 蓝色函数 2 + 蓝色函数 3。每个蓝色函数 (Hard Sigmoid) 在不同位置贡献不同的斜率。

分段线性曲线 (Piecewise Linear Curve)

定义：由多段线段首尾相连组成的曲线（呈锯齿状），每个转折点处的斜率发生变化。

核心性质：

1. 只要取点足够多、位置适当，Piecewise Linear Curve 可以逼近任意连续曲线
 2. 每个 Piecewise Linear Curve 都可以表示为：常数项 + 一组 Hard Sigmoid 之和
- 这意味着：只要有足够多的蓝色函数，理论上可以表达任何复杂的 x 与 y 之间的关系！

2.2 Sigmoid 函数——从离散到平滑

Hard Sigmoid（蓝色函数）的数学表达式写起来不太方便（需要分段定义）。但我们可以用一个平滑的 Sigmoid 函数来逼近它。

Sigmoid 函数

Sigmoid 函数的数学表达式:

$$y = \frac{c}{1 + e^{-(b+wx_1)}} = c \cdot \sigma(b + wx_1)$$

其中 $\sigma(z) = \frac{1}{1+e^{-z}}$ 为 Logistic 函数。“Sigmoid”意为 S 型 (S-shaped), 因其曲线形状似字母 S。

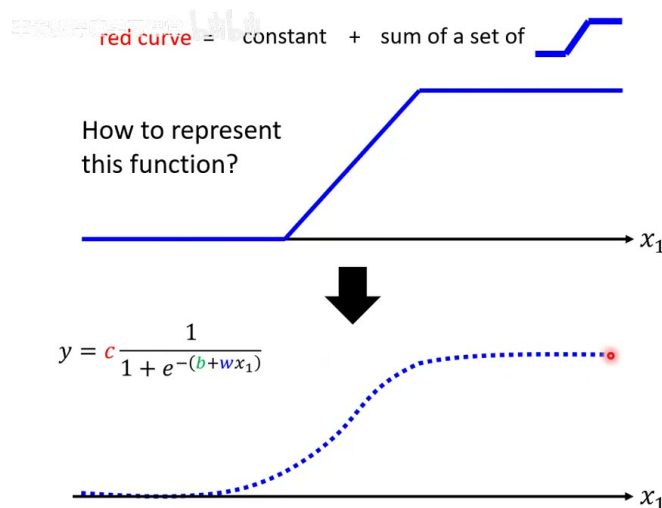


图 3: Sigmoid 函数 $y = c \cdot \sigma(b + wx_1)$: 用平滑的 S 型曲线逼近 Hard Sigmoid。

三个参数分别控制 Sigmoid 曲线的三个特性 (如图 4):

- w —— **控制斜率**: 改变斜坡的陡峭程度
- b —— **控制水平位移**: 将曲线左右移动
- c —— **控制高度**: 改变曲线的幅值

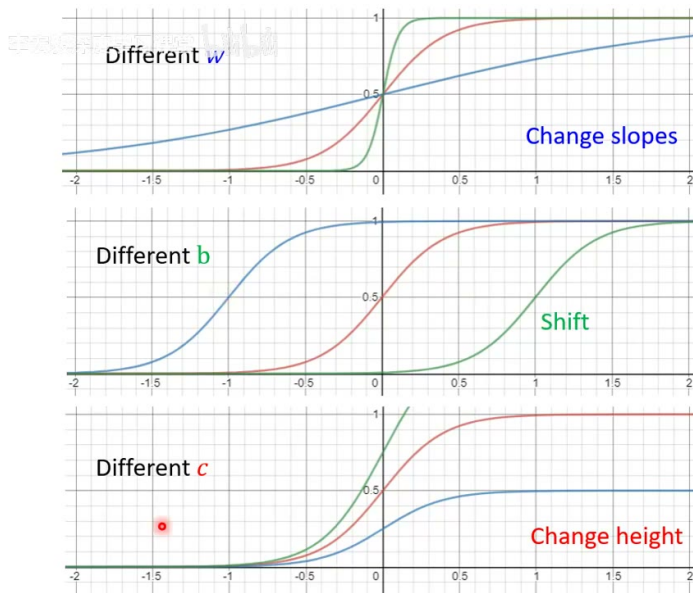


图 4: 参数 w 、 b 、 c 对 Sigmoid 函数形状的影响: w 改斜率, b 改左右位移, c 改高度。

2.3 多特征扩展模型

现在我们可以写出一个带有未知参数、富有弹性的函数:

$$y = b + \sum_{i=1}^n c_i \cdot \sigma(b_i + w_i x_1)$$

对于多个特征 (如: 前 28 天的观看人数), 扩展为:

$$y = b + \sum_{i=1}^n c_i \cdot \sigma\left(b_i + \sum_{j=1}^m w_{ij} x_j\right)$$

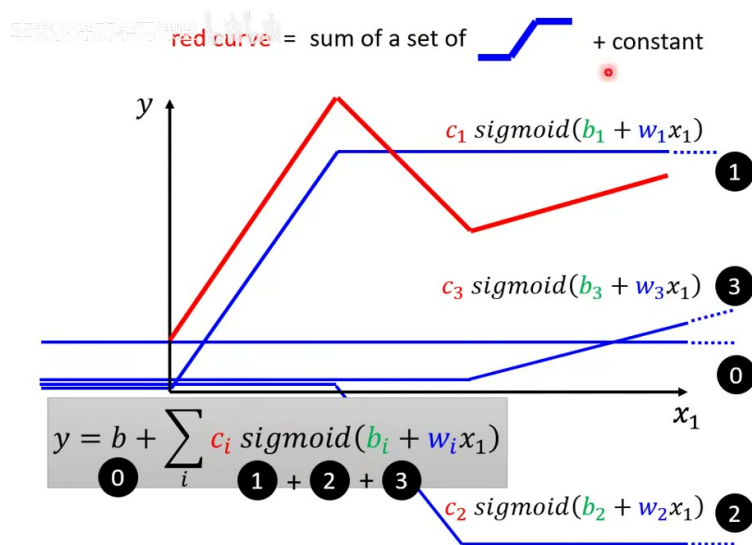


图 5: 多 Sigmoid 模型: $y = b + \sum_i c_i \cdot \sigma(b_i + \sum_j w_{ij} x_j)$ 。不同 c_i 、 b_i 、 w_{ij} 组合出不同形状的 Sigmoid, 叠加后逼近任意连续函数。

其中:

- i : Sigmoid 函数的编号 ($i = 1, 2, \dots, n$)
- j : 特征 (feature) 的编号 ($j = 1, 2, \dots, m$)
- w_{ij} : 第 i 个 Sigmoid 中, 给第 j 个特征的权重
- c_i : 第 i 个 Sigmoid 的缩放系数
- b_i : 第 i 个 Sigmoid 的偏置
- b : 整体的常数项

从线性到非线性的核心思想

线性模型 ($y = b + wx$) $\rightarrow$ 多特征线性模型 ($y = b + \sum_j w_j x_j$) $\rightarrow$ 多 Sigmoid 模型 ($y = b + \sum_i c_i \cdot \sigma(b_i + \sum_j w_{ij} x_j)$)
每一次改进都在增加模型能表达的函数空间, 从而减少 Model Bias。

3 模型的矩阵/向量表示

上面的求和表达式有太多上标下标, 看起来让人头疼。用线性代数可以将其简洁地表达。

3.1 网络结构图

以一个简单的例子说明: 假设有 3 个特征 (x_1, x_2, x_3) 和 3 个 Sigmoid 函数 ($i = 1, 2, 3$)。

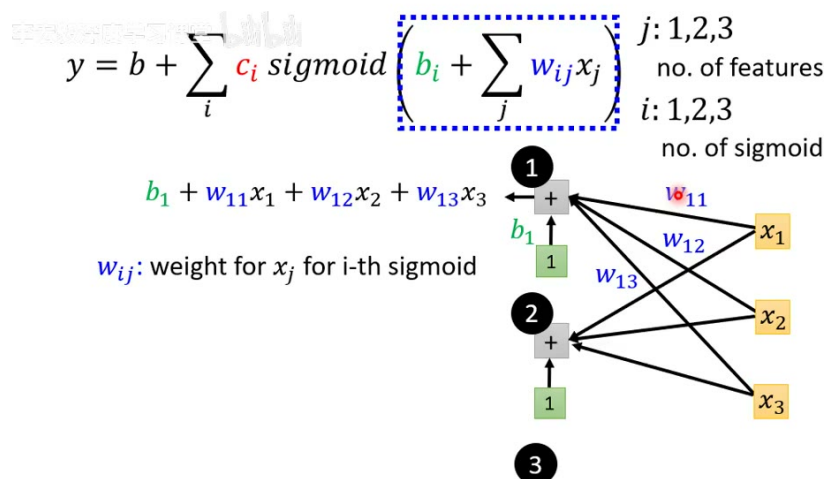


图 6: 模型的图示化表示: 输入 x_1, x_2, x_3 通过不同的权重 w_{ij} 连接到 3 个 Sigmoid 函数, 输出加权求和得到 y 。

每个 Sigmoid 括号里面做的事情是：

$$r_1 = b_1 + w_{11}x_1 + w_{12}x_2 + w_{13}x_3$$

$$r_2 = b_2 + w_{21}x_1 + w_{22}x_2 + w_{23}x_3$$

$$r_3 = b_3 + w_{31}x_1 + w_{32}x_2 + w_{33}x_3$$

然后每个 r_i 通过 Sigmoid 函数： $a_i = \sigma(r_i)$ ，最后输出 $y = b + c_1a_1 + c_2a_2 + c_3a_3$ 。

3.2 线性代数表示

上述运算可以用矩阵和向量简洁表示：

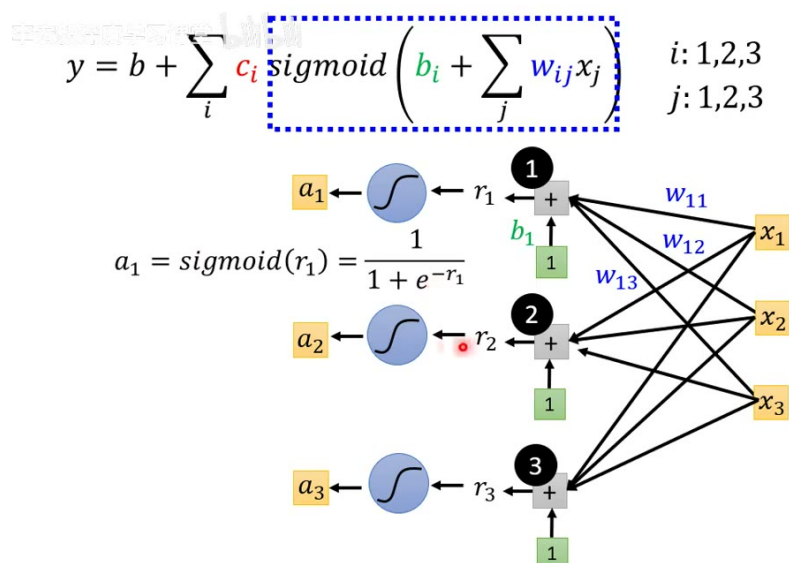


图 7: 线性代数表示： $\mathbf{r} = \mathbf{W}\mathbf{x} + \mathbf{b}$ ， $\mathbf{a} = \sigma(\mathbf{r})$ ， $y = \mathbf{c}^T \mathbf{a} + b$ 。

矩阵形式的完整推导

设输入向量 $\mathbf{x} \in \mathbb{R}^m$ ，有 n 个 Sigmoid 单元：

$$\underbrace{\mathbf{x}}_{m \times 1} \longrightarrow \underbrace{\mathbf{r}}_{n \times 1} = \underbrace{\mathbf{W}}_{n \times m} \mathbf{x} + \underbrace{\mathbf{b}}_{n \times 1} \longrightarrow \underbrace{\mathbf{a}}_{n \times 1} = \sigma(\mathbf{r}) \longrightarrow y = \mathbf{c}^T \mathbf{a} + b$$

将所有未知参数 ($\mathbf{W}$ 、 $\mathbf{b}$ 、 $\mathbf{c}$ 、 b) 拉直拼成一个长向量，记作 $\boldsymbol{\theta}$ ：

$$\boldsymbol{\theta} = [W_{11}, W_{12}, \dots, b_1, b_2, \dots, c_1, c_2, \dots, b]^T$$

注意区分两个 $\mathbf{b}$

图中绿色的 $\mathbf{b}$ 是一个向量 (b_1, b_2, b_3) ，对应每个 Sigmoid 的偏置；灰色的 b 是最后的标量常数项。两者是不同的参数，靠底色区分。

4 损失函数与梯度下降优化

4.1 损失函数 $L(\theta)$

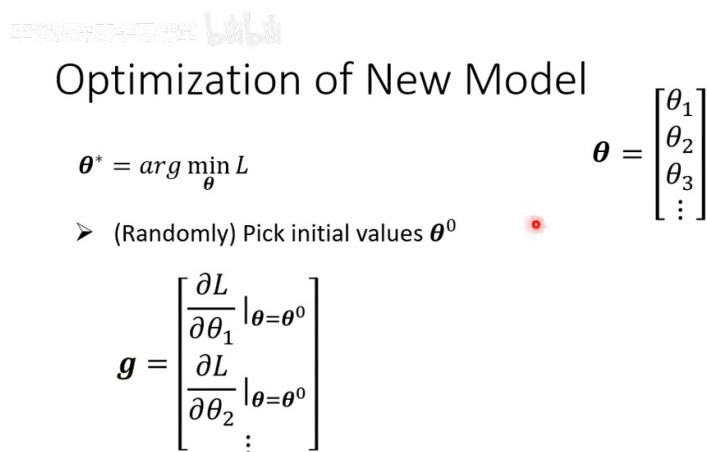
当模型参数很多时，我们不再逐一列出每个参数，而是用 θ 统一表示所有未知参数。损失函数写作 $L(\theta)$ ——问：如果 θ 是某一组数值，这种设定有多不好？

给定一组 θ ：

1. 把参数值代入模型，对每条训练数据计算出预测值 y
2. 计算预测值与真实标签 $\hat{y}$ 之间的差距 $e_n = |y - \hat{y}|$ （或 $(y - \hat{y})^2$ ）
3. 将所有训练数据的误差求和平均： $L(\theta) = \frac{1}{N} \sum_{n=1}^N e_n$

4.2 梯度下降的向量形式

优化目标：找到使 $L(\theta)$ 最小的 θ^* 。



Optimization of New Model

$$\theta^* = \arg \min_{\theta} L$$
$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \end{bmatrix}$$

➤ (Randomly) Pick initial values θ^0

$$g = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} |_{\theta=\theta^0} \\ \frac{\partial L}{\partial \theta_2} |_{\theta=\theta^0} \\ \vdots \end{bmatrix}$$

图 8: 梯度下降更新规则： $\theta^1 = \theta^0 - \eta \cdot g$ ，其中 $g = \nabla L(\theta^0)$ 为梯度向量。

梯度 (Gradient)

梯度 $g = \nabla L(\theta^0)$ 是一个向量，其每个分量是对应参数对 L 的偏导数：

$$g = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_{1000}} \right]^T$$

符号 $\nabla L(\theta^0)$ 表示在 $\theta = \theta^0$ 处计算所有参数对 L 的偏导数，得到梯度向量。

更新规则（向量形式）：

$$\theta^{t+1} = \theta^t - \eta \cdot \nabla L(\theta^t)$$

如果 θ 有 1000 个参数，则 θ^t 、 g 、 θ^{t+1} 都是 1000 维向量。更新逻辑与两个参数的版本完全一致。

4.3 Batch 训练: Epoch 与 Update

在实际训练中, 数据量 N 较大时不会将所有数据一次性算梯度, 而是分组处理:

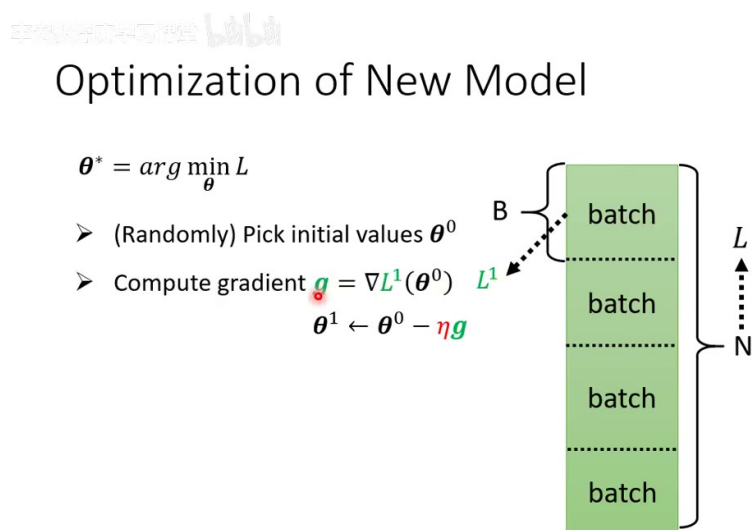


图 9: Batch 训练示意图: 将 N 笔数据随机分成若干 Batch, 每个 Batch 算一次梯度并更新参数。

关键概念区分

- **Batch (批次):** 将全部 N 笔数据随机分成若干组, 每组 B 笔数据称为一个 Batch
- **Update (更新):** 每用一个 Batch 计算梯度并更新一次参数, 即为一次 Update
- **Epoch (轮次):** 把所有的 Batch 都遍历一遍, 即看完所有数据一次, 称为一个 Epoch

例子: $N = 10000$, $B = 10$, 则一个 Epoch 有 $10000/10 = 1000$ 次 Update。

例子: $N = 1000$, $B = 100$, 则一个 Epoch 有 $1000/100 = 10$ 次 Update。

所以当有人说“我做了一个 Epoch 的训练”, 你实际上不知道他更新了几次参数——可以是 1000 次, 也可以是 10 次, 取决于 Batch Size 的大小。

为什么不直接用全部数据算梯度?

全量梯度下降 (Batch GD) 每次迭代都需要计算所有 N 笔数据的梯度, 当 N 很大时计算量极大。使用 Batch 划分后每次只需计算 B 笔数据, 计算更快、也更适合 GPU 并行。更深层的原因 (随机梯度的噪声有助于泛化) 将在后续课程中讲解。

Batch Size B 也是人手动设定的, 属于**超参数** (Hyperparameter)。截至目前, 我们已知的超参数有:

- Learning Rate η (学习率)
- Sigmoid 的数量 n
- Batch Size B

5 ReLU——另一种激活函数

除了 Sigmoid, 还有另一种常见的激活函数——**ReLU** (Rectified Linear Unit, 修正线性单元)。

ReLU 函数

ReLU 的数学表达式:

$$y = c \cdot \max(0, b + wx_1)$$

含义: 取 0 和 $b + wx_1$ 中较大的那个作为输出。

- 若 $b + wx_1 < 0$, 输出为 0
- 若 $b + wx_1 > 0$, 输出为 $b + wx_1$

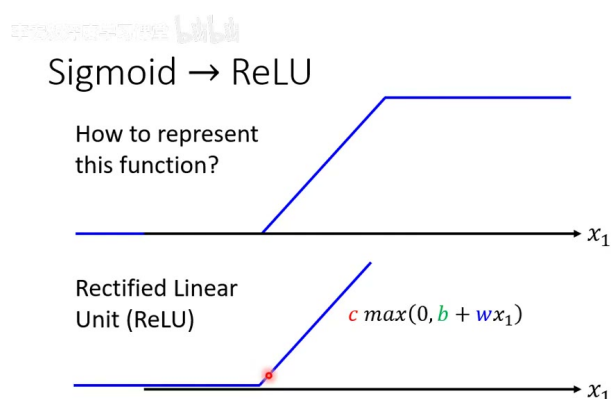


图 10: ReLU 函数: $y = c \cdot \max(0, b + wx_1)$ 。左边水平, 右边斜坡, 形似一条“折线”。

Hard Sigmoid 与 ReLU 的关系: 两个 ReLU 叠加在一起, 就可以构成一个 Hard Sigmoid (即蓝色函数)。

这意味着: 你可以选择用 Sigmoid 来逼近 Hard Sigmoid (间接), 也可以直接用更多的 ReLU 来代替 (直接)。将模型中的 $\sigma(\cdot)$ 替换为 $\max(0, \cdot)$ 即可:

$$y = b + \sum_{i=1}^{2n} c_i \cdot \max\left(0, b_i + \sum_j w_{ij} x_j\right)$$

(需要大约 2 倍的 ReLU 数量来达到与 Sigmoid 相同的表达能力, 因为两个 ReLU 合成一个 Hard Sigmoid。)

Sigmoid 和 ReLU 在机器学习中统称为**激活函数** (Activation Function)。它们是今天最常见的两种激活函数。李宏毅老师在实验中选择 ReLU, 因为“显然 ReLU 比较好”——至于为什么, 将在后续课程揭晓。

6 从浅层到深层——深度学习

6.1 多层堆叠 (Deep Stacking)

我们不需要只在输入和输出之间放一层 Sigmoid/ReLU。可以反复多做几次：

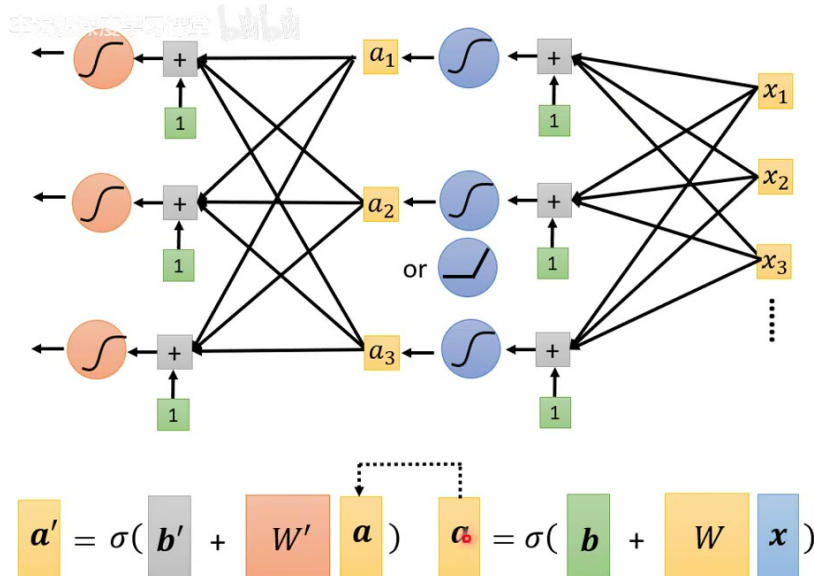


图 11: 深层网络架构: $x \rightarrow a \rightarrow a' \rightarrow \dots \rightarrow y$ 。每一层都有自己的 W 和 b (参数各不相同)。

$$a = \sigma(Wx + b), \quad a' = \sigma(W'a + b'), \quad y = c^T a' + b$$

每次乘上一个新的 W 、加上新的 b 、通过激活函数,就多了一层。做几层? 这又是一个超参数。

6.2 神经网络的命名学

名词起源: 从运算到神经元

- **Neuron (神经元)**: 每个 Sigmoid / ReLU 叫做一个神经元,因为它在形式上被类比为大脑中的神经元
- **Neural Network (神经网络)**: 很多神经元串联起来就是神经网络
- **Hidden Layer (隐藏层)**: 一排神经元叫一层,因为它在输入和输出之间,称为隐藏层
- **Deep Learning (深度学习)**: 有很多隐藏层的网络就叫深度网络,这套技术就叫深度学习

有趣的历史背景: Neural Network 这个名字在 1980–90 年代因为过度吹捧而声名狼藉,写在论文上会被拒稿。后来为了重振旗鼓,换上了“Deep Learning”这个新名字——本质上就是把很多层 Sigmoid/ReLU 堆叠起来的技术。

历史上的深度网络:

- **AlexNet** (2012, 8 层): ImageNet 错误率 16.4%

- VGG (2014, 19 层): 错误率降至 7.3%
- GoogleNet (2014, 22 层): 错误率 6.7%
- ResNet (2015, 152 层): 比台北 101 还高!

7 实验结果

7.1 ReLU 数量与深度的实验

李宏毅老师用真实的 YouTube 频道观看人次数据做了对比实验（输入：前 56 天的观看人数）：

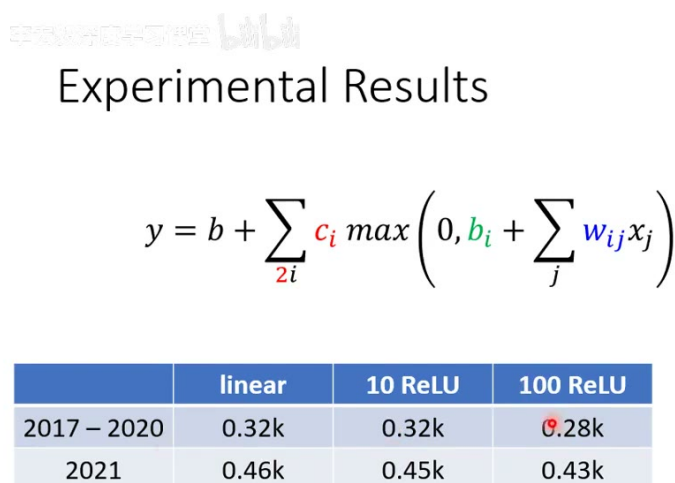


图 12: 不同模型配置在训练数据和测试数据上的 Loss 对比。

表 1: 不同模型在训练集（2020 年）和测试集（2021 年）上的 Loss（单位：千人次）

模型	训练 Loss	测试 Loss	说明
Linear (56 天)	0.32k	0.46k	基线：一条直线
1 层 10 ReLU	~0.32k	~0.46k	10 个神经元不够
1 层 100 ReLU	0.28k	0.43k	显著改善
1 层 1000 ReLU	更低	0.43k (改善不大)	边际效益递减
2 层 (各 100 ReLU)	0.18k	0.42k	深度带来质的飞跃!
3 层 (各 100 ReLU)	0.14k	0.38k	持续进步
4 层 (各 100 ReLU)	0.10k	0.44k	← 过拟合!

核心发现：

1. 从 1 层到 2 层，训练 Loss 从 0.28k 骤降到 0.18k——深度带来了质变
2. 3 层进一步降到 0.14k，测试 Loss 也是最低的 0.38k

3. 4 层在训练集上继续提升 (0.10k), 但测试集反而变差 (0.44k) ——这是**过拟合**!

7.2 预测效果可视化

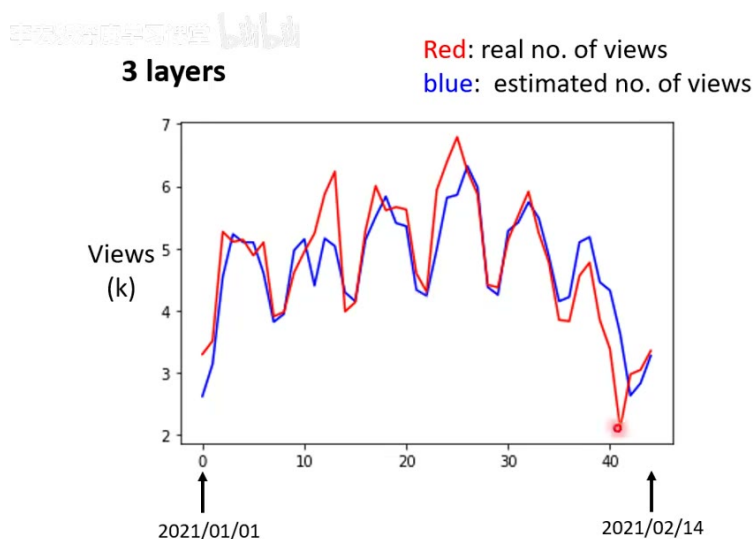


图 13: 3 层网络的预测曲线 (蓝色) 与真实观看人次 (红色) 的对比。模型准确捕捉到了每周的低谷周期, 但未能预测除夕当天的骤降。

从图 13 可以看到:

- 3 层网络准确预测了每个**周五/周六的低谷** (模型学会了周期性模式)
- 但模型**漏掉了除夕当天的低点**, 晚了约一天才预测出低谷
- 这不能怪模型——它只知道前 56 天的观看人数, 不知道那天是除夕

8 过拟合与模型选择

8.1 什么是过拟合 (Overfitting)

过拟合 (Overfitting)

过拟合是指: 模型在训练数据上表现越来越好, 但在没见过的数据 (测试数据) 上表现反而变差的现象。

判断标准: 训练 Loss 下降 $\neq$ 测试 Loss 下降。当训练 Loss 持续改善而测试 Loss 恶化时, 就发生了过拟合。



Why don't we go deeper?

- Loss for multiple hidden layers
 - 100 ReLU for each layer
 - input features are the no. of views in the past 56 days

	1 layer	2 layer	3 layer	4 layer
2017 – 2020	0.28k	0.18k	0.14k	0.10k
2021	0.43k	0.39k	0.38k	0.44k

图 14: 3 层 vs 4 层: 训练 Loss 4 层更好, 但测试 Loss 3 层更好。这就是过拟合——更复杂的模型不一定更好。

8.2 模型选择的原则

假设今天是 2 月 26 日, 我们要预测今天的观看人数。应该选 3 层网络还是 4 层网络?

正确答案是选 3 层。 因为:

- 4 层在训练数据上表现更好 (Loss 0.10k vs 0.14k)
- 但我们在意的是**没见过**的数据 (2 月 26 日)
- 3 层在测试数据上的表现更好 (Loss 0.38k vs 0.44k)
- 所以应该选择在**验证/测试集上表现最好的模型**, 而非训练集

在课堂上大多数同学正确选择了 3 层, 说明大家有很好的直觉。模型选择的系统方法将在下周课程详细讲解。

8.3 关于“为什么是 Deep 而不是 Fat”

一个值得深思的问题: 如果一排足够多的 Sigmoid/ReLU 就可以逼近任意连续函数, 那**为什么**要把网络做深, 而不是简单地把一层做宽 (Fat)?

Deep vs. Fat ——一个开放问题

理论上，一个足够宽的隐藏层确实可以表达任意函数。但实践表明，**深度网络 (Deep)** 比**胖网络 (Fat)** 效果更好。

更深层的原因涉及：

- 层次化的特征提取：浅层学简单模式，深层学抽象概念
- 参数效率：深度结构用更少的参数表达更复杂的函数
- 训练效率与泛化能力

这是后续课程将深入讨论的话题。

9 本章小结

本节承接上节，将一个简单的线性模型逐步演化为我们今天所知的**深度学习**框架：

从线性模型到深度学习的演化路径

Step 1 ——定义模型（逐层演化，模型表达能力依次增强）：

$$\text{线性: } y = b + wx_1$$

$$\text{多特征: } y = b + \sum_j w_j x_j$$

$$\text{非线性: } y = b + \sum_i c_i \cdot \sigma\left(b_i + \sum_j w_{ij} x_j\right)$$

深度： 将上式多层堆叠（每层各有 $\mathbf{W}$ 、 $\mathbf{b}$ ）

Step 2 ——损失函数： $L(\theta)$ ，计算预测值与真实值的差距

Step 3 ——优化： $\theta^{t+1} = \theta^t - \eta \cdot \nabla L(\theta^t)$ （梯度下降，Batch 训练）

本节关键概念清单

概念	英文	说明
模型偏差	Model Bias	模型结构限制导致无法拟合真实关系
分段线性曲线	Piecewise Linear Curve	多段线段组成的曲线，可逼近任意连续函数
Hard Sigmoid	Hard Sigmoid	蓝色函数的形式名称，一个带斜坡的阶跃函数
S 型函数	Sigmoid	$c/(1 + e^{-(b+wx)})$ ，平滑逼近 Hard Sigmoid
激活函数	Activation Function	Sigmoid/ReLU 等非线性函数的统称
修正线性单元	ReLU	$c \cdot \max(0, b + wx)$ ，另一种激活函数
参数向量	θ	所有未知参数拼成的长向量
梯度	Gradient g	所有参数对 Loss 偏导数的向量
批次	Batch	用于梯度计算的一小组数据
轮次	Epoch	遍历全部数据一次
过拟合	Overfitting	训练好但测试差的现象
超参数	Hyperparameter	人手动设定的参数 (η, n, B , 层数等)
隐藏层	Hidden Layer	输入与输出之间的神经元层
深度学习	Deep Learning	有很多隐藏层的神经网络

9.1 下一讲预告

- 为什么 ReLU 比 Sigmoid 好？激活函数的选择
- 如何系统地选择模型（3 层 vs 4 层的决策方法）
- Deep 为什么比 Fat 好？深度学习的理论依据
- 反向传播（Backpropagation）——高效计算梯度的算法

李宏毅老师还提供了一个关于反向传播（Backpropagation）的补充视频链接，它本质上是比一步一步算偏导数更高效的梯度计算方法——与本节讲的梯度下降原理并无不同。

本节完 · 下一节：深度学习训练技巧与模型选择

课程视频：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=2>
